

ПРОЕКТ "МОРСКОЙ БОЙ"

Орлов Юрий

Идея проекта

Проект представляет собой игру морской бой в формате html страницы. Три различных уровня сложности. Морской бой не классический, в данном отсутствуют однопалубные корабли, что позволяет сложному боту играть по теории вероятности, в то время как нормальный бот копирует поведение обычного человека.

Структура проекта

- lobby.html — просто обложка и переходник на игру.
- signin.html и login.html — структура регистрации.
- pole.html — структура основной игровой страницы.
- pole.css — оформление интерфейса игры.
- pole.js — логика игры и интеллект бота.
- другие css и js.
- images — используемые изображения.

```
> images
✓ scripts
  JS get_inf_login.js
  JS get_inf_sign.js
  JS pole.js
✓ styles
  # lobby.css
  # login_signin.css
  # login.css
  # pole.css
  # signin.css
  .gitattributes
  <> lobby.html
  <> login.html
  <> pole.html
  JS server.js
  <> signin.html
```


Lobby

Login

Signin



Регистрация

Username

Email

Password

Password again

Создать

[Уже есть аккаунт? Войти](#)

Boŭmu

Username

Password

[Нет аккаунта? Зарегистрироваться](#)

Boŭmu

Реализация игры:

Отрисовка поля

Функция создает игровое поле программно. Dataset сохраняет координаты клетки для дальнейшей работы

```
const grid = document.getElementById('pole');
const lettersContainer = document.getElementById('cells_name');
const numbersContainer = document.getElementById('cells_number');

const letters = ['A','B','C','D','E','F','G','H','I','J'];
const numbers = ['0','1','2','3','4','5','6','7','8','9'];

function createPole(pole, letterss, numberss) {
  for (let i = 0; i < 10; i++) {
    for (let ii = 0; ii < 10; ii++) {

      const cell = document.createElement('div');
      cell.classList.add('cell');
      pole.appendChild(cell);
      cell.dataset.x_cords = ii;
      cell.dataset.y_cords = i;

    }
  }

  letters.forEach(letter => {
    const span = document.createElement('span');
    span.textContent = letter;
    letterss.appendChild(span);
  });

  numbers.forEach(num => {
    const span = document.createElement('span');
    span.textContent = num;
    numberss.appendChild(span);
  });
}

createPole(grid, lettersContainer, numbersContainer);
```

Функции `allow_pos_x()` и `allow_pos_y()`

Проверяют, можно ли поставить корабль

```
function allow_pos_x(x, y, length)
{
  for (let i = 0; i < length; i++)
  {
    if (x+i > 9 || y > 9) {return false};

    for (let ii = -1; ii <= 1; ii++)
    {
      for (let iii = -1; iii <= 1; iii++)
      {
        const neighbor = getCell(x+i+ii, y + iii);
        if (neighbor && neighbor.state === "ship")
        {
          return false;
        }
      }
    }
  }
  return true;
};
```

```
function allow_pos_y(x, y, length) {
  for (let i = 0; i < length; i++)
  {
    if (x > 9 || y + i > 9) {return false};

    for (let ii = -1; ii <= 1; ii++)
    {
      for (let iii = -1; iii <= 1; iii++)
      {
        const neighbor = getCell(x+ii, y + i + iii);
        if (neighbor && neighbor.state === "ship")
        {
          return false;
        }
      }
    }
  }
  return true;
};
```

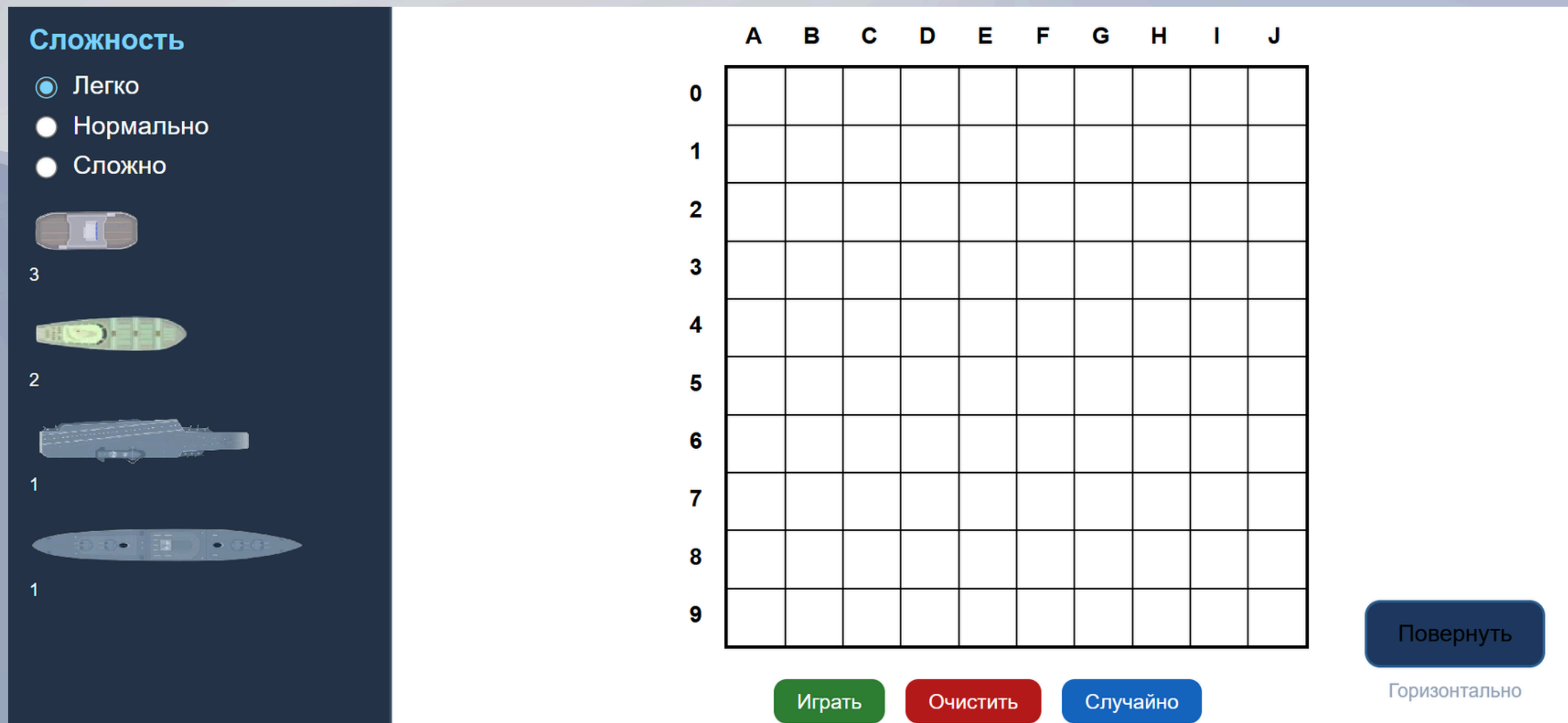

Функции highlight() и unHighlight()

Нужны для подсветки
клеток при наведении.
classList.add() добавляет
CSS-класс
подсветки. Зеленый цвет
показывает допустимую
позицию,
красный цвет показывает
ошибку установки.

```
function highlight(x, y, length, valid) {  
  unHighlight();  
  for (let i = 0; i < length; i++)  
  {  
    const Cell = document.querySelector(`.cell[data-x_cords="${x + i}"][data-y_cords="${y}"]`);  
    if (Cell)  
    {  
      Cell.classList.add(valid ? "highlight_on" : "highlight_of");  
    }  
  }  
};  
  
function highlight_y(x, y, length, valid) {  
  unHighlight();  
  for (let i = 0; i < length; i++)  
  {  
    const domCell = document.querySelector(`.cell[data-x_cords="${x}"][data-y_cords="${y + i}"]`);  
    if (domCell)  
    {  
      domCell.classList.add(valid ? "highlight_on" : "highlight_of");  
    }  
  }  
};  
  
function unHighlight() {  
  document.querySelectorAll(".cell").forEach(cell => {  
    cell.classList.remove("highlight_on", "highlight_of");  
  });  
};
```

Расстановка кораблей

Отрисовка изображения кораблей происходит через DOM, размещение через dragleave и dragover. Реализация кнопок играть, очистить, случайно, повернуть. Также выбор сложности



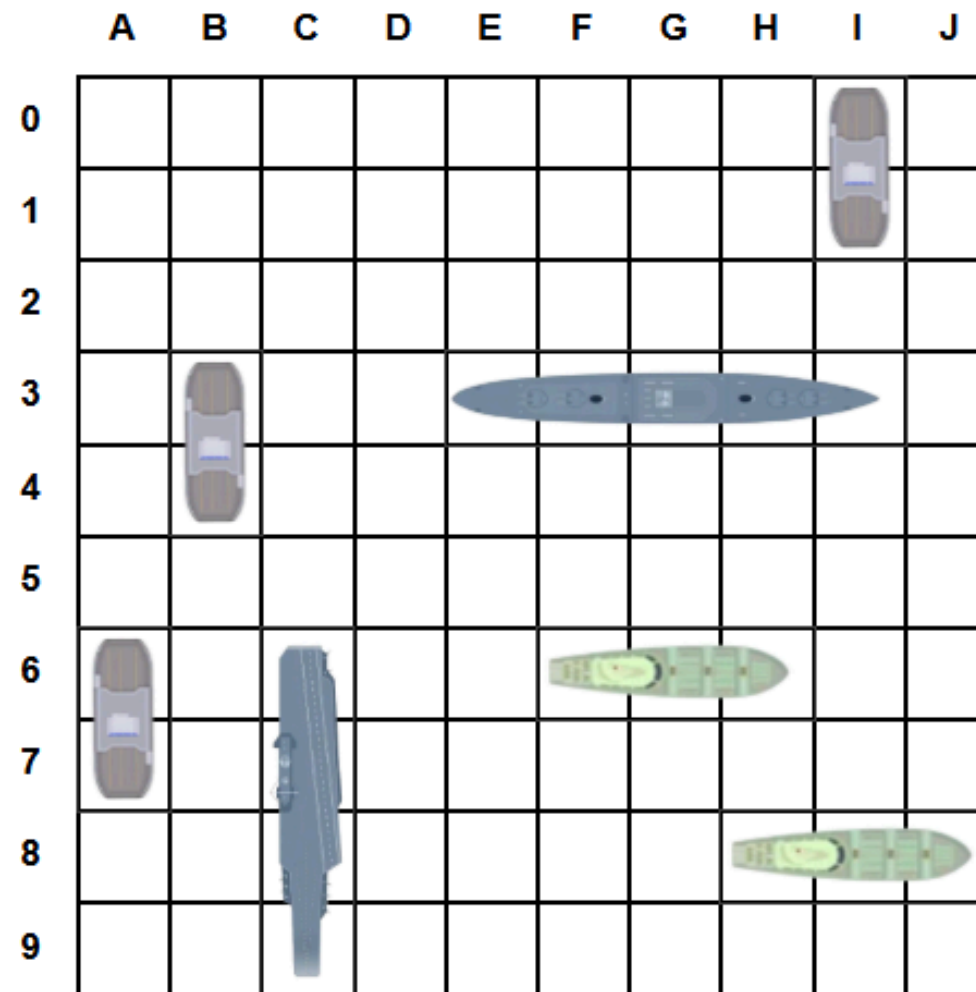
Начало игры

Сначала через DOM
убирается левая панель
и кнопки.

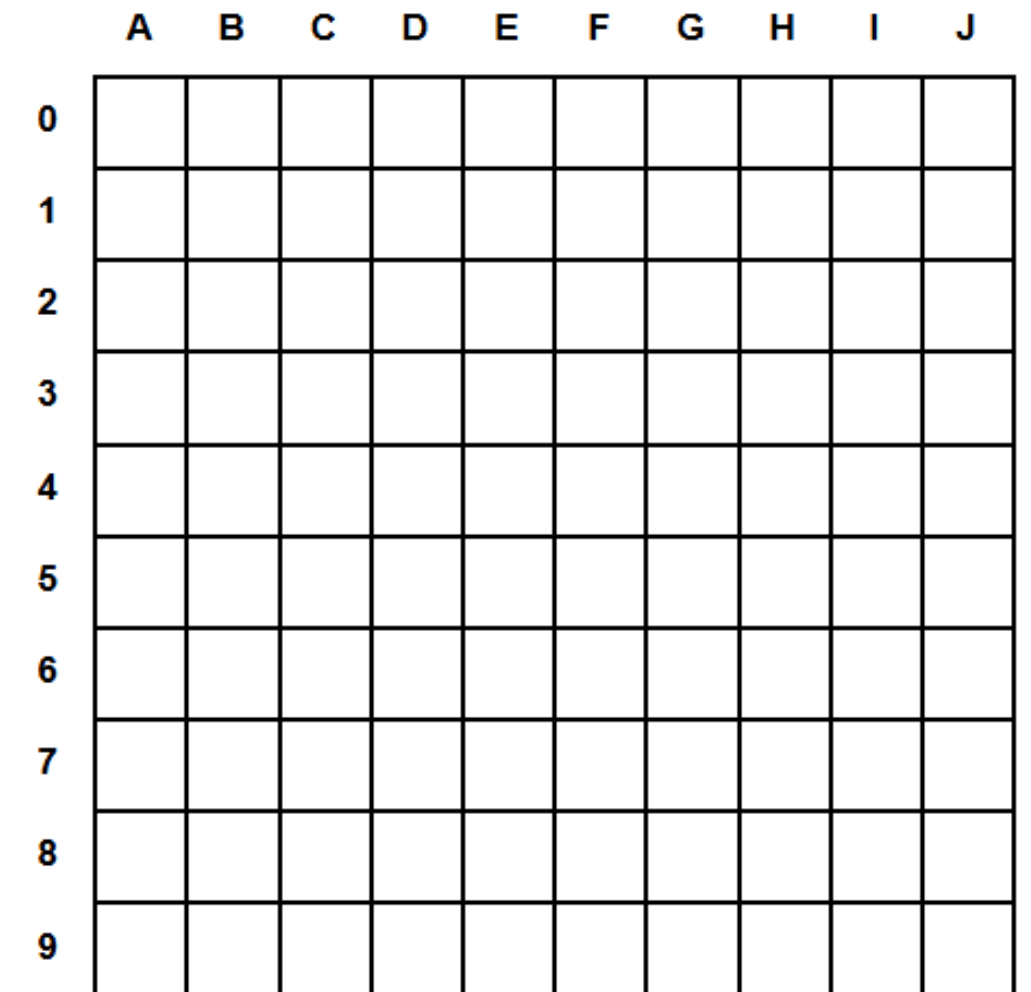
Отрисовывается поле
бота , для него
копируются основные
функции. Убирается
возможность
перетаскивать корабли
и убирать их.
Расстановка кораблей
бота.

Ваш ход

Ваше поле



Поле противника



Сама игра

**В зависимости от выбранной сложности
начинается игра:**

- **Лёгкий:** стреляет в случайные клетки в независимости от того, попал он или нет.
- **Нормальный:** также случайно стреляет, но в случае попадания начинает добивать корабль.
- **Сложный:** копирует поведение нормального, но после того, как будет открыто более 20% поля, начинает высчитывать наиболее возможные места расположения кораблей.

Ваше поле

[illegible]

Поле противника

[illegible]

The background features a complex, abstract pattern of overlapping, semi-transparent geometric shapes, primarily triangles and polygons, in a palette of soft blues, purples, pinks, and greens. These shapes radiate from a central point near the top center, creating a sunburst or starburst effect. The overall composition is dynamic and modern.

Спасибо за внимание!